A Dynamic Behavior Perspective on Flaky Tests

Anonymous Author(s)

ABSTRACT

Understanding the behavior of automated tests is crucial for improving software development practices. For instance, the flakiness phenomenon of tests that pass and fail intermittently without changes may undermine the trust in continuous integration results. There exists a constant effort by practitioners and researchers to introduce approaches and tools to detect, debug, and fix flaky tests. While there are initiatives to comprehend their nature and root causes, the literature lacks deeper analyses of the dynamic behavior of flaky tests.

This paper presents an empirical study conducted to shed some light on the dynamic behavior of flaky tests. To do so, we implemented three dynamic analyses that collect pieces of information across multiple executions of flaky tests related to: *(i)* code coverage, *(ii)* profiling, and *(iii)* function-level tracing. Such analyses were performed using 247 flaky tests from 61 open-source Python projects.

The results show that flaky tests exhibit heterogeneous dynamic behaviors across the three analyses. While most flaky tests present distinct code coverage and function-level tracing profiles between passed and failed executions, a smaller subset preserves identical structural behaviors despite alternating outcomes. Profiling analysis also reveals differences between passed and failed executions, including cases in which code coverage and function-level tracing remain unchanged. These findings provide new insights into the dynamic behavior of flaky tests and may support future research on flaky test analysis, prediction, debugging, and mitigation.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**.

KEYWORDS

Automated tests, flaky tests, dynamic analysis, tracing

ACM Reference Format:

Anonymous Author(s). 2026. A Dynamic Behavior Perspective on Flaky Tests. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Tests play a crucial role in evaluating quality and reliability of software systems [28], relying on automated techniques to uncover bugs before they reach production [16]. Test cases may be written as code scripts that verify, in continuous integration (CI) environments, the expected behavior of the ever-evolving *System Under Test* (SUT), mitigating the threats of introducing new bugs or breaking changes. As noted by Garousi and Felderer [7], automated testing offers numerous benefits across various projects, including repeatable, predictable, and efficient test executions. However, it is also a tedious process that is prone to implementation errors and requires substantial investment. Even when excluding the initial implementation cost, the ongoing maintenance of automated tests is necessary, as they must evolve alongside the production code.

Automated testing enables a constant and trustworthy delivery of new software versions to the end users, but this process may be challenged by the presence of flaky tests. Habchi et al. [12] define a flaky test as the one that may pass or fail unpredictably across different executions, resulting in non-deterministic outcomes that confuse developers interpreting test results. When the flakiness' root cause is explained by the influence of previously executed tests, a flaky test is called *order-dependent* (OD) [26]. All other flaky tests are referred to as *non-order-dependent* (NOD) [21].

According to Listfield [25], test flakiness has become a significant concern, as ignoring flaky tests could AVG overlooking bugs in production — thus failing to provide effective test coverage. He reports that Google's CI system executes approximately 4.2 million tests, of which around 63,000 are identified as flaky each week. Although this represents only 2% of all tests, it is still a considerable number and poses challenges for developers. Flaky tests may impact negatively the development time, disrupt CI, undermine good testing practices, and ultimately reduce system quality [12]. Software developers have also manifested concerns about test flakiness via surveys [5, 8, 32, 35]. For instance, Eck et al. [5] notice that flaky is a recurring issue, affecting 20% of participants monthly, 24% weekly, and 15% daily. This frequency underscores the persistent presence of the problem throughout the software development cycle. Furthermore, the study found that 91% of developers encountered flaky tests at least a few times per year, with 56% rating the issue as moderate and 23% describing it as severe.

Flaky tests have received a lot of attention from practitioners and researchers [25, 27, 29, 32, 37, 40]. While there exist research initiatives on understanding flaky tests from the developer's perspective, code commits, and test reruns, there is room for improvement in studying flaky tests from a dynamic viewpoint. Dynamic information may be more sensitive for characterizing flaky test execution [14]. Static metrics may guide prediction models to classify a test always as flaky. A drawback of considering dynamic information is the cost of rerunning and the overhead imposed by the analyses. For the former, an industrial study brings evidence that the cost of rerunning tests is negligible and inexpensive [24], and developers are less worried about such costs [8]. For the latter, recent initiatives have shown that dynamic analyses can be implemented with small or near-zero overhead [6, 34].

This paper aims to evaluate the dynamic behavior observed in flaky tests. We developed three analyses that, at runtime, collect

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2026 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

dynamic information across many runs, from the perspective of code coverage, profiling, and function-level tracing. The results are based on 247 real-world flaky tests extracted from 61 open-source Python projects. We observed that [results summarized here].

The main contributions of this paper are threefold. First, our study sheds some light on the dynamic behavior of flaky tests. This knowledge may support further advances in the detection, debugging, and fixing of flaky tests. Second, the design and implementation of the dynamic analyses are publicly available, and allow for new extensions. Finally, all flaky tests used in the study are provided as open resources for future replication.

The remainder of this paper is organized as follows. Section 2 introduces a motivating example. Section 3 presents the study setup, while Section 4 analyzes the results. Section 5 discusses the main results and limitations. The related work is presented in Section 6, and Section 7 provides the conclusion.

2 MOTIVATING EXAMPLE

The Python code in Figure 1 illustrates the motivating example: the function under test is presented in lines 1-23, while an automated test case written using the testing framework pytest[1] is in lines 24-38. This example is based on a flaky test we detected while conducting this study, it is extracted from the pycco[2] project, a tool that generates HTML documentation from Python code using a literate programming style [18]. The function under test, `get_language` (lines 1-23), is responsible for determining the programming language of a given source file. It receives two parameters: `source`, the path of the file, and `code`, its actual source code content (line 7). If `source` is provided (line 8), the function first attempts to infer the language from its file extension: it extracts the extension using `os.path.basename` together with a regular expression (line 9), and if the extension is a key in the `supported_languages` dictionary (line 10), the corresponding entry is returned immediately (line 11). If `source` is not provided, or its extension is not recognized, the function falls back on the pygments[3] library, using `lexers.guess_lexer` to infer the language directly from the content of `code` (line 14); the guessed lexer name is then compared, case-insensitively, against the names registered in `supported_languages` (lines 15-16). If a match is found, the corresponding entry is returned (line 17); otherwise, a `ValueError` exception is raised (line 19). This exception, along with any `ValueError` that pygments itself might raise when it fails to identify a suitable lexer for `code`, is caught (line 20) and re-raised with the standardized message `"Can't figure out the language!"` (line 23). The test (lines 31-38) aims to assert the behavior of function `p.get_language`, imported in line 28. This function receives two strings as input, the first is the path of a file f, and the second is the source code of f; then it returns the programming language of f using both arguments; otherwise, it launches a `ValueError` exception if cannot decide the language. The function is verified twice, (i) it should return `PYTHON` with a valid Python code in line 35, and (ii) it should raise a `ValueError` exception with an invalid language code (lines 37-38). The flakiness of the test in Figure 1 stems from a *deadline* setting of Hypothesis[4], which bounds the wall-clock execution time allowed for each call of the decorated test to 200 ms by default. In one observed failure, Hypothesis generated `source=None` and this call took 266.86 ms, exceeding the deadline, so the example was reported as falsifying; when Hypothesis re-executed the very same input to confirm the failure, the call completed in 3.28 ms, well within the deadline, and no failure was reproduced. Since a falsifying example that cannot be reproduced on replay violates Hypothesis's own consistency check, it raises a `Flaky` error rather than a regular assertion failure.

Alternating outcomes alone do not explain how the internal execution differs between them; characterizing this difference is the goal of the three dynamic analyses this study will adopt. Since passed and failed executions of `test_get_language_bad_source` may traverse different branches of `get_language`—extension-based early return versus pygments-based inference—we use *code coverage*, measured as *Coverage Percentage*, the proportion of the project's source code exercised by the test run; a change in this proportion between outcomes indicates a different code path.

These branches also differ in computational cost: a dictionary lookup by file extension is comparatively cheap, whereas invoking pygments's `guess_lexer` can have a significant performance impact, since it calls the `analyse_text` method of every registered lexer class and returns the one with the highest score. We therefore use *profiling*, with two metrics: `ncalls`, the number of times each monitored function is called during a run, which reveals whether `guess_lexer` was invoked; and `cumtime_top20`, the cumulative time spent in a function and its callees, computed for the twenty functions with the highest cumulative time per run; a timing difference indicates a costlier branch was taken.

Coverage and profiling provide only aggregated, line- or function-level summaries, so we complement them with *function-level tracing*, which records the complete call hierarchy of an execution; we measure it via `Tracing_Size_Bytes`, the size, in bytes, of the resulting trace file, which grows with the number and depth of recorded calls. Together, these three complementary perspectives—code coverage, profiling, and function-level tracing—allow us to move beyond the pass/fail outcome and characterize the dynamic behavior underlying test flakiness, detailed in the remainder of this paper.

3 STUDY SETUP

This study aims to characterize flaky tests from a dynamic behavior perspective. Therefore, we define the following research questions.

- **RQ1** To what extent does code coverage vary between executions of flaky tests?
- **RQ2** To what extent do profiling metrics vary between executions of flaky tests?
- **RQ3** To what extent does function-level tracing vary between executions of flaky tests?

[1] `https://pytest.org`
[2] `https://github.com/pycco-docs/pycco`
[3] `https://pygments.org`
[4] Hypothesis is a property-based testing library for Python, it allows a test to describe the range of valid inputs and automatically generates and explores many values within that range, including edge cases that might otherwise go untested. See `https://github.com/hypothesisworks/hypothesis`.

Code for the function under test

```python
import os
import re
from pygments import lexers

from pycco.languages import supported_languages

def get_language(source, code):
    if source:
        m = re.match(r'.*(\..+)', os.path.basename(source))
        if m and m.group(1) in supported_languages:
            return supported_languages[m.group(1)]

    try:
        language_name = lexers.guess_lexer(code).name.lower()
        for entry in supported_languages.values():
            if entry["name"] == language_name:
                return entry
        else:
            raise ValueError()
    except ValueError:
        # If pygments can't find any lexers, it will raise its own
        # subclass of ValueError.
        raise ValueError("Can't figure out the language!")
```

Code for automated test case

```python
import pytest
from hypothesis import given
from hypothesis.strategies import none, text

import pycco.main as p


@given(text() | none())
def test_get_language_bad_source(source):
    valid_code = "#!/usr/bin/python\n def foo():\n return True"

    assert p.get_language(source, valid_code) == PYTHON

    with pytest.raises(ValueError) as e:
        p.get_language(source, "invalidlangcode")
```

Figure 1: Motivating example.

- **RQ4** What behavioral patterns emerge in flaky tests that exhibit variability across dynamic analyses?

RQ1 intends to take a deeper look at the potentially changing behavior of code coverage between multiple runs of a known flaky test, while **RQ2** approaches this perspective using profiling data. Then, **RQ3** assesses the dynamic behavior of flaky tests using function-level tracing. Finally, **RQ4** investigates the behavior and code structures observed in flaky tests that show variability across the dynamic analyses used in previous RQs.

3.1 Selection of Flaky Tests

In this study, we focus on NOD flaky tests, as they occur more frequently than OD ones and remain less explored in the literature [22]. To identify and label the NOD flaky tests, we relied on two sources: for projects reproduced from Gruber et al. dataset, we adopted the classification provided by the FlaPy tool [11]; for projects sourced from Parry et al., we used the NOD labels already provided in their dataset [30].

Initially, we sought to reproduce the NOD flaky tests reported in the large-scale study by Gruber et al. [11], which analyzes open-source Python projects from PyPI[5]. To ensure the reliability of our experiments, we established rigorous inclusion criteria: we retained only those projects for which we could install the dependencies, build the environment, and reproduce the flaky behavior. Among these, the flaky tests for four projects—`pycco`, `forex-python`, `textdistance`, and `pyquery`—were sourced from another study that applied a similar methodology based on FlaPy.

To expand our dataset beyond the scope of Gruber et al. [11], we incorporated additional Python repositories from other research sources. Specifically, we selected 15 projects from the dataset provided by Parry et al. [30] that also contain NOD flaky tests. After applying our inclusion criteria, one project (*electrum*) had to be excluded due to build failures caused by obsolescence. Consequently, our final dataset comprises 61 projects in total: 43 from Gruber et al. [11] , 14 from Parry et al. [30], and 4 additional projects.

Table 1 shows pieces of information related to the projects with flaky tests analyzed: project name, commit hash of the version used, the number of lines of Python code (LoC)[6], the number of test cases (#TCs), and the number of flaky tests (#FTs). In total, our dataset comprises 247 flaky tests distributed across 61 different open-source projects. While there is an average of 4.05 flaky tests per project, the distribution is not uniform. More than half of the dataset (34 projects, or 55.7%) contains exactly one flaky test. Conversely, a few projects concentrate a large portion of the observations, with the `airflow` project acting as a major outlier containing 66 flaky tests, followed by `rs3clans.py` (28) and `prefect` (25).

3.2 Data Collection

To collect pieces of data for the three dynamic analyses, we implemented a plugin for pytest. Some Python code from FlaPy was also reused to support test executions. The plugin basically enables a particular analysis before a test execution, and produces a log file with raw data afterwards. Data collection was organized into four execution rounds, each executing the full set of 247 previously identified flaky tests 100 times: a baseline round without any instrumentation (*No Dynamic Analysis* – NDA), and three additional rounds, each applying one of the three dynamic analyses under study.

For *code coverage*, we reused the functionalities of Coverage.py[7]. This tool enables the systematic measurement of line coverage, with the goal of assessing the extent to which a test exercises the source code and identifying lines not exercised by the test suite. In this round, each of the 247 tests was executed 100 times, and, from each execution, we collected the *Coverage Percentage* (%), i.e., the overall proportion of the project's source code lines exercised by that specific test.

To obtain *profiling* data, we used the embedded Python profiler cProfile[8]. This module identifies performance bottlenecks by measuring, for each function executed, statistics such as the number of calls, execution time, and call frequency. We collected two metrics: `ncalls`, the number of times a given function was called during

[5]The Python Package Index – `https://pypi.org`
[6]Data collected with cloc.
[7]`https://coverage.readthedocs.io`
[8]`https://docs.python.org/3/library/profile.html`

Table 1: Overview of the projects.

Project Name[1]	Hash	LoC	#TCs	#FTs	Project Name[1]	Hash	LoC	#TCs	#FTs
Gruber et al.									
amphipathic	b4d2d49	458	5	1	piripherals	74e4807	2,497	85	1
analytic_shrinkage	752a0f9	199	5	2	pybnn	6bd2774	2,039	17	2
cuckoopy	e5fab0c	266	18	1	pycco	27b1bf0	615	17	2
darkchess	c626de5	1,421	9	2	pylabeledrf	23e2c2b	473	2	2
data-partitioner	c37bca0	234	18	1	pymannkendall	081a1de	1,689	14	1
dockerfile_bakery	fc53175	601	8	3	pymemorydb	9c8e864	902	47	1
fistro	42c129d	352	8	1	pypfm	78cccd1	143	5	1
forex-python	e1fa196	620	39	7	pyquery	f2533d1	2,173	150	1
forkliftpy	d0b878e	172	5	1	pyss3	0522e0d	5,309	18	1
fossor	d8dbdc4	2,475	63	1	pysshutil	e3ab876	1,517	57	2
ftpsync	bbf40b8	410	1	1	python-matlab-functions	2860bf4	123	4	2
gamble	d17b258	635	14	1	python-timemarker	16fc4d4	262	8	1
hotwing-core	f4d864b	3,816	49	1	r_map	56cad81	1,617	66	1
jinja2-github	4012e5f	58	2	2	rs3clans.py	692ffb8	389	36	28
kb_python	3ce4d8e	6,519	129	1	service-manager	e0a9f2a	4,121	57	2
keplertools	8a46aea	590	14	3	setigen	30acd46	1,166	10	2
kevinarpe-rambutan3	5d74699	7,253	190	1	spreading_dye_sampler	4282f76	217	5	1
kindle-maker	5c478bd	390	6	1	textdistance	bcd2174	2,726	430	4
langsplit	d30d47b	249	11	1	torch-embed-sim	ace404f	76	1	1
layouts-python	0b036e1	414	10	1	torch-layer-normalization	89f405b	151	5	1
levitate	f6b2b26	5,083	1,446	5	uhashring	e45d6a7	671	20	1
nnweaver	e7a94d7	1,563	29	1	Verifone	5509c08	1,106	30	1
override	e34bd3c	261	5	1	wiki-futures	ae6bec3	97	6	1
packeteer	2c6e0e8	1,316	50	3					
Parry et al.									
airflow	c743b95	1,848,983	3,458	66	loguru	f31e971	216,984	1,255	4
Flexget	4fb2508	71,028	1,335	1	prefect	6b59d98	540,192	7,038	25
fonttools	f3a9030	306,348	3,456	1	requests	a1a6a54	429,626	537	5
hypothesis	ccac436	1,089,353	4,386	5	seaborn	703259f	22,439	1,028	1
ipython	95d2b79	465,536	846	6	setuptools	4d64156	36,554	704	1
kombu	2aeb732	293,063	1,025	2	urllib3	ba95e9e	295,316	1,900	15
libcloud	449b315	147,811	9,840	3	xonsh	e9b12c8	666,014	4,782	9

[1]The project names have links to the corresponding GitHub pages.

the test execution, and `cumtime_top20`, the cumulative time spent in a function and all functions it calls, measured from invocation to return, computed for the twenty functions with the highest cumulative time in that execution.

To implement function-level tracing, we utilized Python's native `sys` module—specifically the `sys.settrace` tracing API[9], which intercepts function calls, exceptions, and line-by-line execution to build a hierarchical record of functions called during a test and their return values; we chose this approach due to its availability in the standard library. We collected the `Tracing_Size_Bytes`—the size, in bytes, of the resulting trace file.

Given a certain test analyzed in our experiments, there are two groups of passing and failing test executions. We treated each group as a sample and computed the average (AVG), the standard deviation (SD), and the coefficient of variation (CV). SD and CV represent the variation measures we are interested in analyzing in our RQs.

3.3 Experimental Setup

All experiments were conducted on a workstation equipped with an Intel Core i9-14900KF processor (base clock of 3.2 GHz), 64 GB of DDR5 RAM (5600 MHz), and a 1 TB NVMe solid-state drive. The operating system was Linux Mint 22.1, running kernel version 6.8.0-110-generic on an x86-64 architecture. Python 3.12 was used to execute the test suites and the dynamic analysis instrumentation.

Each test was executed 400 times according to the procedure described in Section 3.2. The executions were performed separately for coverage, profiling, and tracing, with 16 CPU workers allocated to the experimental workload in each analysis.

4 ANALYSIS OF RESULTS

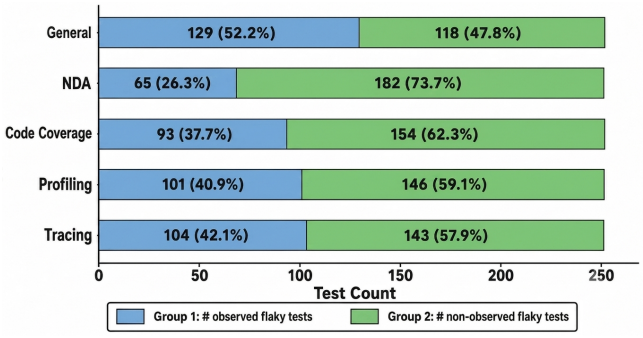


Figure 2: Breakdown of flaky test observations.

Figure 2 presents a breakdown of flaky test observations among the 247 tests evaluated under each instrumentation approach. The

[9]`https://docs.python.org/3/library/sys.html`

Table 2: Groups according to Code Coverage.

Group	Flakiness behavior	AVG Behavior	Variation?		Count	(%)
			Passed	Failed		
#1	Observed	Different	No	No	37	15.0
#2	Observed	Different	Yes	Yes	22	8.9
#3	Observed	Different	No	Yes	14	5.7
#4	Observed	Different	Yes	No	8	3.2
#5	Observed	Equal	No	No	12	4.9
#6	Observed	Equal	Yes	Yes	0	0.0
#7	Observed	Equal	No	Yes	0	0.0
#8	Observed	Equal	Yes	No	0	0.0
#9	No (Only Passed)	-	No	-	81	32.8
#10	No (Only Passed)	-	Yes	-	38	15.4
#11	No (Only Failed)	-	-	No	31	12.6
#12	No (Only Failed)	-	-	Yes	4	1.6
Total					**247**	**100.0**

General (Total) category represents the union of all tests classified as an observed flaky test in at least one of the three dynamic analyses, revealing a total of 129 tests (52.2%).

Under *No Dynamic Analysis* (NDA), 65 tests (26.3%) were classified as observed flaky tests. Under *Code Coverage*, the count was 93 tests (37.7%). Under *Profiling* – which is more intrusive, as it measures temporal execution data – the count was 101 tests (40.9%). Under *Tracing* – the most resource-intensive analysis, due to function-level interception – the count was 104 tests (42.1%).

4.1 RQ1: Code Coverage

One possible explanation for a flaky test is that different executions exercise distinct parts of the code, influenced by timing, external state, concurrency, or randomness. Code coverage records which lines are exercised during a given execution, making it a proper tool for investigating this hypothesis, which corresponds to RQ1.

Table 2 shows 12 groups identified when observing the code coverage, the flakiness behavior associated with the tests, and variation. For each group (1st column), the last two columns show the number of flaky tests ("Count"), and the percentage, respectively. Column "Flakiness behavior" indicates whether we observed flakiness in the group or not, while column "AVG behavior" indicates whether the average code coverage values are different or equal between passed and failed executions. Finally, column "Variation" indicates if variation (SD > 0) is observed in both passing and failing executions (columns "Passed" and "Failed"). We use character '-' to indicate that cell is not applicable, e.g., group #9 has only flaky tests with passing executions, so values in columns "AVG behavior" and "Failed" are not presented.

Of the eight possible combinations for the observed flaky tests, five groups (groups #1-#5) totaling 93 tests (37.7%) occurred, while groups #6-#8 did not. When analyzing *flaky tests with different coverage when passing and failing*, Group #1 with 37 tests (15.0%) had no variation; from a code coverage perspective, their execution was deterministic, covering the same amount of lines in all runs. On the other hand, Group #2 with 22 tests (8.9%) had varying code coverage in both passing and failing runs, while Group #3 with 14 tests (5.7%) had variation only in failed runs, and Group #4 had variation only in passed runs: 8 tests (3.2%). In Group #5, 12 tests (4.9%) had *equal coverage when passing and failing*, without variation.

The remaining 154 tests (62.4%) in groups #9-12 yielded only a single result across the 100 runs and were classified as *non-observed flaky tests*. Group #9 (passed only, no variation) comprises 81 tests (32.8%), making it the largest one. Group #10 (only passed, with variation) comprises 38 tests (15.4%). Group #11 (only failed, without variation) comprises 31 tests (12.6%). Finally, Group #12 (only failed, with variation) comprises 4 tests (1.6%).

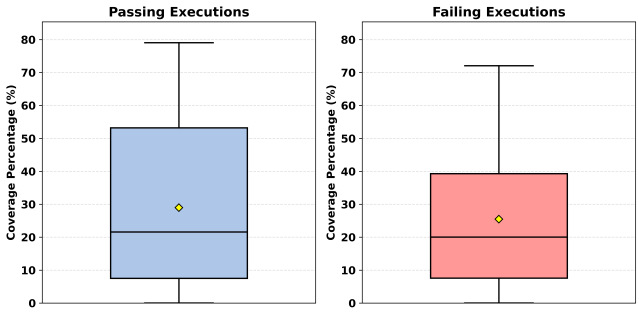
Tests with varying coverage: Groups #2, #3, #4, #10, and #12 comprise every test whose coverage varied in at least one of its observed outcomes. Together, they account for 86 tests (34.8%): Group #10 alone, at 38 tests (15.4%), is larger than Groups #2, #4, #6, and #8 combined (30 tests, 12.1%), meaning most passing-side variation occurs in tests where no failure was observed at all. Group #2 (22 tests, 8.9%) and Group #4 (8 tests, 3.2%) are the only Observed Flaky Tests with variation on the passing side. On the failing side, Group #12 accounts for only 4 tests (1.6%) – far fewer than Group #10's 38 tests on the passing side – indicating that unstable coverage is much less common among tests observed only as failing than among tests observed only as passing.

Passing tests with varying coverage: Groups #2, #4, and #10 comprise every test whose coverage varied across its passed executions, regardless of whether a failed outcome was also observed. Together, they account for 68 tests (27.5% of the 247 tests). Group #10 alone, at 38 tests (15.4%), is larger than Groups #2 and #4 combined (30 tests, 12.1%), meaning most passing-side variation occurs in tests where no failure was observed at all.

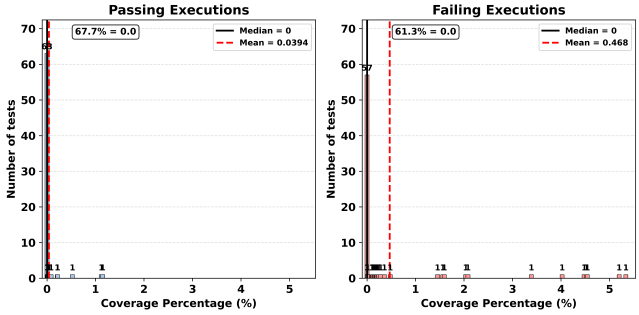
Figure 3 presents the distribution of the average (AVG) and standard deviation (SD) of code coverage; to make a paired comparison for both passing and failing executions, we take into account only the 93 observed flaky tests. Notice that an SD value of 0.0 is possible as it indicates an absence of variation across multiple runs.

Regarding the AVG, presented in Figure 3a as boxplots, passed executions show an interquartile range of 7.52% to 53.21% and a median of 21.59%, with whiskers ranging from 0.02% to 79.07% and no outliers; the AVG of 28.98% lies above the median. Failed executions show an interquartile range of 7.58% to 39.30% and a median of 20.03%, with whiskers ranging from 0.02% to 72.09% and, likewise, no outliers; the AVG of 25.49% is also above the median. The medians (21.59% vs. 20.03%) and AVG's (28.98% vs. 25.49%) are close across the two outcomes, indicating that the overall coverage level does not appear to have noticeable differences.

Regarding the SD, shown as histograms in Figure 3b, the majority of tests show no variation in code coverage. In passed executions, 63 out of 93 tests (67.7%) show an SD of 0.0%, while the remaining 30 tests range from 0.001% to a maximum of 1.15%, with only 2 tests reaching 1% or higher variation. In failed executions, 57 tests (61.3%) show an SD of 0.0%, while the remaining 36 tests range from 0.003% to a maximum of 5.338%, with 12 tests reaching 1% or higher. The SD median is 0.0% for both outcomes, and the AVG exceeds the median due to the non-zero values (0.039% for passed executions and 0.468% for failed ones). Therefore, failed executions show a smaller proportion of tests with no variation than passed ones (61.3% vs. 67.7%), and when variation occurs, it reaches higher values.



(a) AVG for coverage values among observed flaky tests.



(b) SD for coverage values among observed flaky tests.

Figure 3: Distribution of AVG (a) and SD (b), restricted to the 93 observed flaky tests, for the Coverage Percentage metric. In each panel, passed executions are shown on the left and failed executions on the right. In (a), the box shows the interquartile range, the horizontal line the median, and the diamond the AVG. In (b), the solid line marks the median and the dashed line the AVG.

Answer to RQ1: When comparing passing and failing executions, most observed flaky tests (87.1%) exhibit differences in code coverage, although these differences are relatively small. The remaining 12.9% exhibit identical code coverage in both passing and failing executions. When analyzing multiple executions with the same test outcome, approximately one-third of the tests (34.8%) exhibit some variation in code coverage.

4.2 RQ2: Profiling

The outcome of a flaky test may also depend on how much computational work a given execution performs, rather than only on which lines it executes—for instance, an extra retry or a call to a costlier code path can change the number of function calls or the time spent without necessarily changing which branches are covered. Profiling records these execution-cost metrics for every function invoked during a run, making it a direct tool for investigating this hypothesis, which corresponds to RQ2.

As with Coverage Percentage, we computed the AVG, standard deviation, and coefficient of variation for each of the ten collected profiling metrics, separately for the passed and failed executions

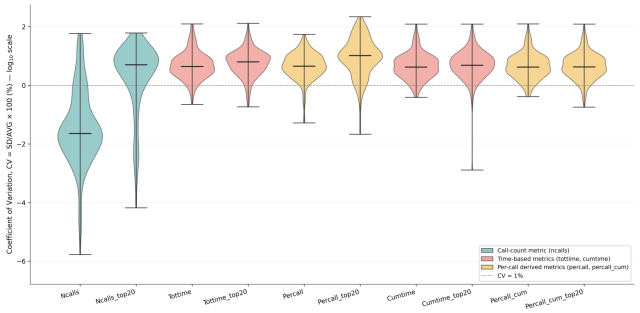


Figure 4: Distribution of the coefficient of variation (CV) across profiling attributes for the 101 observed flaky tests, pooling passed and failed executions.

of the 101 observed flaky tests. Figure 4 shows the resulting CV distribution for all ten metrics, pooling passed and failed executions.

Ncalls is the number of times a given function was called during a test execution. *Cumtime_top20* is the cumulative time, in seconds—the same unit used by the other time-based profiling metrics collected in this study—that a function and all the functions it calls spent executing, measured from invocation to return, computed for the twenty functions with the highest such value in each execution. These two metrics exemplify two distinct regimes within the CV distribution shown in Figure 4. *Cumtime_top20* has a median CV of 4.16%, with 65.8% of its values falling between CV = 1% and CV = 10%, indicating that accumulated execution time varies at a moderate, comparable magnitude across most executions. *Ncalls* has a median CV of 0.02%, with 87.6% of its values below CV = 1%, indicating that the number of function calls is deterministic for most executions; its maximum value (58.58%) is nonetheless approximately 3.5 orders of magnitude above its median, compared to approximately 1.5 orders of magnitude for *Cumtime_top20* (from 4.16% to its maximum of 124.02%)—the widest such span among the ten metrics. We selected these two metrics as representative of these regimes: *Cumtime_top20* for a moderate, near-constant level of variation present in most executions, and *Ncalls* for a variation that is absent from most executions but, when present, departs from its typical value by a wider margin than any other metric.

Flaky tests with different AVG when passing and failing: Of the eight possible combinations for Observed Flaky Tests, Groups #1–#4 (different AVG) account for 101 tests (40.9%) for both *Cumtime_top20* and *Ncalls*, while Groups #5–#8 (equal AVG) did not occur for either metric. Within Groups #1–#4, Group #2 (variation in both outcomes) is by far the largest: 79 tests (32.0%) for *Cumtime_top20* and 77 tests (31.2%) for *Ncalls*. The two metrics diverge slightly in Groups #1 and #3: Group #1 (no variation in either outcome, yet different AVG) registers 1 test (0.4%) for *Ncalls* against none for *Cumtime_top20*; Group #3 (variation only in failed executions) registers 5 tests (2.0%) for *Ncalls* against 4 (1.6%) for *Cumtime_top20*. Group #4 (variation only in passed executions) is identical for both metrics, at 18 tests (7.3%).

Flaky tests with equal AVG when passing and failing: Groups #5, #6, #7, and #8 did not occur for either *Cumtime_top20* or *Ncalls*.

Table 3: Distribution of groups according to Profiling.

Group	Test Type	AVG	Variation		cumtime_top20		ncalls	
			Passed	Failed	Count	(%)	Count	(%)
#1	Observed	Different	No	No	0	0.0	1	0.4
#2	Observed	Different	Yes	Yes	79	32.0	77	31.2
#3	Observed	Different	No	Yes	4	1.6	5	2.0
#4	Observed	Different	Yes	No	18	7.3	18	7.3
#5	Observed	Equal	No	No	0	0.0	0	0.0
#6	Observed	Equal	Yes	Yes	0	0.0	0	0.0
#7	Observed	Equal	No	Yes	0	0.0	0	0.0
#8	Observed	Equal	Yes	No	0	0.0	0	0.0
#9	Only Passed	-	No	-	0	0.0	0	0.0
#10	Only Passed	-	Yes	-	126	51.0	126	51.0
#11	Only Failed	-	-	No	0	0.0	0	0.0
#12	Only Failed	-	-	Yes	20	8.1	20	8.1
Total					**247**	**100.0**	**247**	**100.0**

Non-observed flaky tests: The remaining 146 tests (59.1%) yielded only a single outcome across the 100 runs under profiling instrumentation. Group #9 (only passed, no variation) and Group #11 (only failed, no variation) contain 0 tests for either metric. Group #10 (only passed, with variation) comprises 126 tests (51.0%), the largest group in the table, and Group #12 (only failed, with variation) comprises 20 tests (8.1%) – identical counts for both *Cumtime_top20* and *Ncalls*.

Tests with varying profiling metrics: Groups #1, #5, #9, and #11 are the only groups corresponding to a test with no variation in any observed outcome; for *Cumtime_top20*, all four are empty, meaning every one of the 247 tests (100.0%) shows variation in at least one of its observed outcomes. *Ncalls* shows the same pattern with a single exception: Group #1 contains 1 test, so 246 tests (99.6%) vary in at least one outcome. Within this near-total variation, Group #10 is the single largest contributor for both metrics (126 tests, 51.0%), meaning most of this variation occurs in tests observed only as passed.

Answer to RQ2: Profiling metrics differentiate execution outcomes for all 101 observed flaky tests. Internal variation occurs in both outcomes for 79 tests (78.2%) under Cumtime_top20 and 77 tests (76.2%) under Ncalls. While Cumtime_top20 exhibits moderate, consistent variation (median CV of 4.16%), Ncalls is highly deterministic (median CV of 0.02%) but produces extreme magnitude outliers when variation does occur.

4.3 RQ3:Function-Level Tracing

Beyond which code is exercised (coverage) and how much computational work is performed (profiling), a flaky test's executions may also differ in the sequence and depth of function calls performed—for instance, an execution that returns early invokes fewer, shallower calls than one that falls through to further processing. Function-level tracing records this complete call hierarchy for every execution, making it a direct tool for investigating this hypothesis, which corresponds to RQ3.

Table 4 shows 12 groups identified when observing function-level tracing, the flakiness behavior associated with the tests, and

Table 4: Distribution of groups according to function-level Tracing.

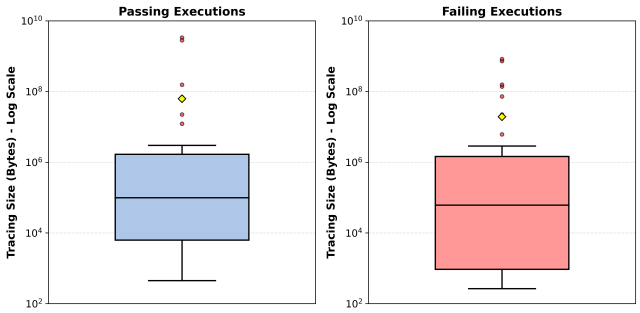
Group	Test Type	AVG Behavior	Variation		Count	(%)
			Passed	Failed		
#1	Observed	Different	No	No	20	8.1
#2	Observed	Different	Yes	Yes	48	19.4
#3	Observed	Different	No	Yes	19	7.7
#4	Observed	Different	Yes	No	11	4.5
#5	Observed	Equal	No	No	6	2.4
#6	Observed	Equal	Yes	Yes	0	0.0
#7	Observed	Equal	No	Yes	0	0.0
#8	Observed	Equal	Yes	No	0	0.0
#9	No (Only Passed)	-	No	-	51	20.6
#10	No (Only Passed)	-	Yes	-	62	25.1
#11	No (Only Failed)	-	-	No	17	6.9
#12	No (Only Failed)	-	-	Yes	13	5.3
Total					**247**	**100.0**

variation. For each group (1st column), the last two columns show the number of flaky tests ("Count"), and the percentage, respectively. Column "Test Type" indicates whether we observed flakiness in the group or not, while column "AVG behavior" indicates whether the average tracing size values are different or equal between passed and failed executions. Finally, column Variation" indicates if variation (SD > 0) is observed in both passing and failing executions (columns "Passed" and "Failed"). We use character '-' to indicate that cell is not applicable, e.g., group #10 has only flaky tests with passing executions, so values in columns "AVG behavior" and "Failed" are not presented.

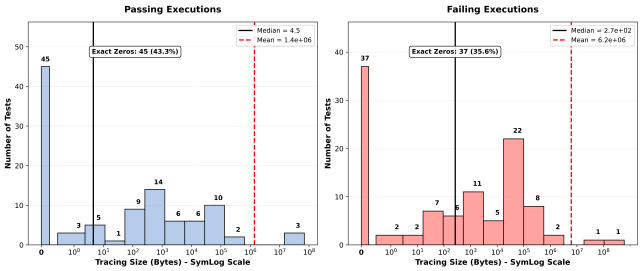
Of the eight possible combinations for the observed flaky tests, five groups (groups #1-#5) totaling 104 tests (42.1%) occurred, while groups #6-#8 did not. When analyzing *flaky tests with different trace size (measured in bytes) when passing and failing*, Group #1 with 20 tests (8.1%) had no variation; from a function-level tracing perspective, their execution was deterministic, recording the same trace size in all runs. On the other hand, Group #2 with 48 tests (19.4%) had varying trace size in both passing and failing runs, while Group #3 with 19 tests (7.7%) had variation only in failed runs, and Group #4 had variation only in passed runs: 11 tests (4.5%). In Group #5, 6 tests (2.4%) had *equal trace size when passing and failing*, without variation.

The remaining 143 tests (57.9%) in groups #9-#12 yielded only a single result across the 100 runs and were classified as *non-observed flaky tests*. Group #9 (passed only, no variation) comprises 51 tests (20.6%). Group #10 (only passed, with variation) comprises 62 tests (25.1%), making it the largest group in the table. Group #11 (only failed, without variation) comprises 17 tests (6.9%). Finally, Group #12 (only failed, with variation) comprises 13 tests (5.3%).

Tests with varying trace size: Groups #2, #3, #4, #10, and #12 comprise every observed flaky test whose trace size varied in at least one of its observed outcomes. Together, they account for 91 tests (36.8%): Group #10 alone, at 62 tests (25.1%), is larger than Groups #2, #4, #6, and #8 combined (59 tests, 23.9%), meaning most passing-side variation occurs in tests where no failure was observed at all. Group #2 (48 tests, 19.4%) and Group #4 (11 tests, 4.5%) are



(a) AVG for passed and failed executions (log scale).



(b) SD distribution of Tracing Size (SymLog scale).

Figure 5: Distribution of the AVG and SD of Tracing Size (Bytes) across the 104 observed flaky tests, comparing passed and failed executions. (a) Boxplots of non-zero AVG sizes on a logarithmic scale. (b) SymLog histograms of SD sizes, highlighting exact zero-dispersion tests (43.3% for passed and 35.6% for failed) alongside log-binned non-zero variations.

the only Observed Flaky Tests with variation on the passing side. On the failing side, Group #12 accounts for only 13 tests (5.3%) – far fewer than Group #10's 62 tests on the passing side – indicating that unstable trace size is much less common among tests observed only as failing than among tests observed only as passing.

Passing tests with varying trace size: Groups #2, #4, and #10 comprise every test whose trace size varied across its passed executions, regardless of whether a failed outcome was also observed. Together, they account for 121 tests (49.0% of the 247 tests). Group #10 alone, at 62 tests (25.1%), is larger than Groups #2 and #4 combined (59 tests, 23.9%), meaning most passing-side variation occurs in tests where no failure was observed at all.

Figure 5 presents the distribution of the average (AVG) and standard deviation (SD) of tracing size (in bytes); to make a paired comparison for both passing and failing executions, we take into account only the 104 observed flaky tests. Notice that an SD value of 0.0 is possible as it indicates an absence of variation across multiple runs.

Regarding the AVG, presented in Figure 5(a) as boxplots, passed executions show an interquartile range of 6,579 to 1,457,442 bytes and a median of 72,944 bytes, with the upper whisker extending to 3,008,641 bytes and 19 extreme outliers reaching up to 7,634,933,056 bytes; the AVG lies significantly above the median, characterizing a heavily right-skewed distribution driven by these extreme values. Failed executions show an interquartile range of 623 to 1,344,148 bytes and a median of 34,240 bytes, with the upper whisker reaching 3,304,185 bytes and 7 outliers extending up to 829,883,875 bytes; similarly, the AVG is positioned far above the median. While the overall ranges are comparable, passed executions tend to exhibit a higher median (72,944 vs. 34,240 bytes) and reach much more extreme upper outliers, indicating that tracing sizes tend to be larger when tests pass. Regarding the SD, shown as SymLog histograms in Figure 5(b), a substantial portion of the tests exhibits no variation in either outcome. In passed executions, 45 out of 104 tests (43.3%) show an exact SD of 0.0 bytes, while the remaining 59 tests exhibit non-zero variations spreading across multiple orders of magnitude, up to 10^8 bytes. In failed executions, a smaller proportion of 37 out of 104 tests (35.6%) show an SD of 0.0 bytes, with the remaining 67 tests also spreading up to the 10^8 magnitude order. For both outcomes, the AVG heavily exceeds the median due to the extreme non-zero variations (1.4×10^6 AVG vs. 4.5 median for passed executions, and 6.2×10^6 AVG vs. 270 median for failed ones). Therefore, failed executions show a smaller proportion of tests with absolutely no variation compared to passed ones (35.6% vs. 43.3%), and when variation does occur, they demonstrate a higher central tendency (median of 270 vs. 4.5 bytes) and a considerably larger AVG dispersion.

> *Answer to RQ3: When comparing passing and failing executions, most observed flaky tests (94.2%) exhibit differences in function-level tracing, although these differences are relatively small. The remaining 5.8% exhibit identical tracing sizes in both passing and failing executions. When analyzing multiple executions with the same test outcome, approximately one-third of the tests (36.8%) exhibit some variation in tracing size.*

4.4 RQ4: Observed Cases

> *Answer to RQ4:*

5 DISCUSSION

5.1 Implications for Researchers and Practitioners

- Meaningful number of flaky tests (one-third) has varying code coverage: [ToDo]
- Differences in code coverage between passing and failing executions are pretty small: With a witness run in both cases, the developer would need to debug a few lines of code. [ToDo]
- Coverage and tracing have similar results.

The results of this study provide empirical evidence that flaky tests do not exhibit a single dynamic behavior. Instead, different execution patterns were observed across the three dynamic analyses considered in this work. While most flaky tests showed distinct code coverage and function-level tracing profiles between passed and failed executions, other cases remained structurally unchanged despite alternating outcomes. These observations reinforce the complexity of flaky tests and suggest that different dynamic analyses capture different aspects of their execution behavior.

For researchers, our findings contribute a broader characterization of the dynamic behavior of flaky tests. Rather than focusing on a single runtime perspective, this study investigates code coverage, profiling, and function-level tracing under the same experimental setting, showing that these analyses reveal complementary execution patterns. We expect that these empirical observations may support future studies investigating the dynamic characteristics of flaky tests and the design of new techniques for flaky test analysis.

For practitioners, the results indicate that dynamic analyses can provide additional information beyond the pass/fail outcome observed during repeated executions. In particular, differences observed in code coverage, profiling metrics, and function-level tracing may help characterize how flaky tests behave under different execution outcomes. Such information can complement existing debugging activities by providing additional evidence about the runtime behavior associated with flaky executions.

Finally, the identification of flaky tests that preserve identical code coverage and function-level tracing despite alternating between passed and failed outcomes suggests that different dynamic analyses may reveal different aspects of flaky behavior. These observations reinforce the importance of investigating flaky tests from multiple dynamic perspectives when characterizing their runtime behavior.

5.2 Threats to Validity

As with any empirical study, our research is subject to certain limitations that must be acknowledged. We categorize these threats into three main areas:

- **Construct Validity:** This threat concerns whether our chosen metrics adequately represent the dynamic behavior of flaky tests. To mitigate this, we utilized a combination of established metrics—code coverage, uncompressed trace size, and profiling data (such as *ncalls* and cumulative execution time). While these metrics serve as robust proxies for structural and footprint analysis, they may not capture lower-level hardware or operating system interruptions that could also characterize dynamic execution.
- **Internal Validity:** Internal validity relates to uncontrolled factors that might influence the experimental results. Dynamic execution is highly sensitive to environmental noise, state leakage, and external dependencies. To strictly isolate our observations and ensure that the captured divergences were inherent to the tests themselves, all experimental executions were rigorously conducted within isolated, clean virtual environments (venvs).
- **External Validity:** This threat pertains to the generalizability of our findings. Our empirical evaluation was conducted exclusively on a dataset of Python projects. Given the specific characteristics of the Python interpreter (e.g., Global Interpreter Lock, dynamic typing), the behavioral patterns we observed might differ in other ecosystems. Consequently, future research is necessary to replicate this multi-perspective analysis in compiled and strongly typed languages, such as Java or C++, to confirm the universality of these dynamic profiles.

Overall, this study provides an empirical characterization of flaky tests from three complementary dynamic perspectives: code coverage, profiling, and function-level tracing. The results show that flaky tests exhibit multiple dynamic execution patterns rather than a single behavioral profile across repeated executions. We believe that these findings contribute to a better understanding of the dynamic behavior of flaky tests and may support future studies on flaky test analysis, prediction, debugging, and mitigation.

6 RELATED WORK

Comprehending flaky tests. The test flakiness phenomenon has been investigated from different perspectives in the literature. In a pioneer paper, Luo et al. [26] analyze 201 commits that likely fix flaky tests in 51 open-source Apache projects. The main root causes observed are related to async wait, concurrency, and test order dependency. For these three main causes, the authors investigate manifestation strategies, and common fixing approaches.

Other studies have used similar empirical methods by mining repositories and historical data from open-source projects [4, 10, 15, 36, 38]. Such methods have also been applied to industrial projects. For instance, Lam et al. [20] investigate the lifecycle of flaky tests in six large-scale proprietary projects at Microsoft. The authors observed that asynchronous calls are the leading cause, while providing insights into recurrence, runtime of flaky tests, and the time-before-fix. In other study with industry, Leinen et al. [24] investigate the cost factors behind flaky tests in CI development processes. The authors proposed a cost model, instantiating it in a case study with a large-scale industrial software project. The results indicate that rerunning failed tests has negligible cost, which is way cheaper than manually debugging flaky tests.

Another line of research is the human perception about the test flakiness. The most widespread strategy is to apply online surveys with developers [5, 8, 32, 35]. Eck et al. [5] surveyed 21 Mozilla developers about nature, origin (test or production code), fixing efforts about flaky tests, while fixing strategies were retrieved from code repositories. They confirm the prevalence of seven flakiness types presented by Luo et al. [26], while introducing four new types: too restrictive range, test case timeout, platform dependency, and test suite timeout. Then, an online survey was conducted with 121 participants, showing that flaky tests are frequent, with consequences in resource allocation, scheduling, and test suite reliability. The main challenges reported are related to the failing context reproduction, and understanding the flakiness nature.

Gruber and Fraser [8] surveyed a greater sample with 335 professionals. Their results show that 51% of participants have contact with flaky tests weekly, 66% classify it as a moderate or serious issue. Concurrency-related is the most common cause, and three new root causes were identified: uninitialized variables, compiler differences, and undocumented API changes. Loss of trust and waste of developer time are the main consequences. Mitigation strategies are mainly rerunning and rewriting tests, and developers want more support for static detection, better visualizations, and more educational opportunities.

Qualitative studies with semi-structured or open-ended interviews are less extensively adopted. Ahmad et al. [2] interviewed participants from five different case companies about the root

causes of flaky tests and their perception of harmfulness. And Habchi et al. [12] conducted semi-structured interviews with 14 practitioners about where flakiness is located, its perceived impact, how it is addressed, and mitigation measures.

Other studies present analyses about flaky tests using empirical data collected from multiple reruns of automated tests. Gruber et al. [11] investigate the prevalence of flaky tests in 22,352 open-source Python projects. To do so, the authors developed a tool called FlaPy that was used to rerun the projects' test suites 400 times. Their study identified 7,571 flaky tests (0.86% of all tests), caused mostly by order-dependency (59%) and infrastructure (28%). Among NOD flaky tests, the main causes are network, randomness, and IO.

Lam et al. [22] investigate flaky tests in Java projects, based on 4,000 reruns using the default test runner. They found 107 flaky tests and studied their characteristics from order-dependent and non-order-dependent perspectives. They found out that up to five reruns may detect most of the flaky tests, running tests in different orders with fewer times is more effective, executing tests in isolation is better for debugging, and debugging first tests that fail together.

Parry et al. [33] analyze the co-occurrences of flaky tests failures using the FlakeFlagger dataset [3] which involves log data of 10,000 test suite reruns from 24 Java projects. The study observes the widespread and significant presence of systemic flakiness, clusters of flaky tests with co-occurring failures sharing the same root causes. The researchers noticed that 75% of the flaky tests are in a cluster, static test case distances along with ML models can be used to predict systematic flakiness, and the root causes of clusters are mainly related to networking, followed by external dependency.

Dynamic information in flaky tests. Machine Learning (ML) models have been researched to predict flaky tests [17]. Although there exists a prevalence of statically collected features (e.g., token vocabulary, LoC, code metrics, smells) [14], the usage of dynamic information has been taken into account. King et al. [17] employ features related to test results like test execution time, failure rate and flip rate. FlakeFlagger [3] and Wang et al. [39] use a diverse set of features based on code coverage, such as class coverage, line coverage, and coverage of lines modified in the last commits (i.e., covered changes). Built on top of previous studies, Parry et al. [31] introduce a set of features, called Flake16, to train ML models to detect flaky tests. Among the Flake16 features, dynamic features like peak number of threads, write counts, and read counts are more impactful in the model's decisions. They also consider the number of voluntary context switches and peak memory usage.

Unlike classifying flaky tests, Lampel et al. [23] introduce an approach that classifies CI jobs that fail as actual software bugs or flakiness in the job. The classifiers are based on features extracted from job telemetry data, like runtime and CPU load. In a recent initiative, Haben et al. [14] argue that the flakiness responsibility should be placed in the test execution, not in the test itself like in previous studies. In this vein, dynamic features may be more adequate once they can better characterize the state of a test execution.

Debugging flaky tests also makes use of dynamic information. Lam et al. [19] use function-level instrumentation to collect runtime properties to identify root causes. Habchi et al. [13] apply traditional Spectrum-Based Fault Localization to pinpoint program classes that are linked to the flaky behavior of a specific test. They rely on a single run to collect coverage information. Gruber and Fraser [9] introduce Spectrum-based Flaky Fault Localization (SFFL) with the aim of identifying the exact source code statement that provokes the test flakiness. By adapting well-known fault localization techniques, the authors adapt spectrum-based formulas that take the multi-coverage behavior of flaky tests into account. With their dataset, they observed that around 80% of flaky tests present varying coverage between different runs. Flakiness Debugger [41] uses a custom algorithm, called DIVERGENCE, that shows the first divergence between passing and failing runs of a flaky test. To do so, it needs to collect full execution traces (the order in which lines are covered). FlakyPy [1] also builds on the DIVERGENCE algorithm but considers values and anomalies related to local variables.

Concerning the state-of-the-art on comprehending test flakiness, the literature lacks more studies that characterize flaky tests from a dynamic behavior viewpoint. In particular, we could not find studies that assess in-depth the three dynamic analyses discussed here. While dynamic information has been widely adopted to support approaches and tools for flaky test detection, debugging and fixing, we believe that our study can help to better incorporate dynamic analyses related to code coverage, profiling, and function-level tracing into future approaches and tools.

7 CONCLUDING REMARKS

In this paper, we have presented an empirical study with the goal of shedding some light on the dynamic behavior of flaky tests. The study focuses on three dynamic perspectives, code coverage, profiling, and function-level tracing.

The results indicate. Across the investigated dimensions, our findings reveal that test flakiness is a multi-faceted phenomenon that cannot be captured by a single dynamic perspective. While code coverage (RQ1) and function-level tracing (RQ3) demonstrate that most flaky behaviors stem from structural control-flow divergences, profiling analysis (RQ2) shows that flaky also heavily impacts execution footprint and resource overhead. Crucially, our qualitative investigation (RQ4) ***completar depois...*** . Together, these results demonstrate that test flakiness operates along a spectrum, ranging from explicit path alterations to hidden execution overheads.

In future, we plan to expand the sample of flaky tests analyzed, and how the results relate to non-flaky tests. Another direction is to investigate if the insights collected from the dynamic analyses can support the engineering of new features for flaky test prediction. Finally, we hope that the results and insights can help to advance novel approaches for test flakiness detection, debugging, and fixing.

REFERENCES

[1] Azeem Ahmad, Erik Norrestam Held, Ola Leifler, and Kristian Sandahl. 2022. Identifying Randomness related Flaky Tests through Divergence and Execution Tracing. In *15th IEEE International Conference on Software Testing, Verification and Validation Workshops ICST Workshops 2022, Valencia, Spain, April 4-13, 2022*. IEEE, 293–300. https://doi.org/10.1109/ICSTW55395.2022.00057

[2] Azeem Ahmad, Ola Leifler, and Kristian Sandahl. 2021. Empirical analysis of practitioners' perceptions of test flakiness factors. *Softw. Test. Verification Reliab.* 31, 8 (2021). https://doi.org/10.1002/STVR.1791

[3] Abdulrahman Alshammari, Christopher Morris, Michael Hilton, and Jonathan Bell. 2021. FlakeFlagger: Predicting Flakiness Without Rerunning Tests. In *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid, Spain, 22-30 May 2021*. IEEE, 1572–1584. https://doi.org/10.1109/ICSE43902.2021.00140

[4] Jonathan Bell, Owolabi Legunsen, Michael Hilton, Lamyaa Eloussi, Tifany Yung, and Darko Marinov. 2018. DeFlaker: automatically detecting flaky tests. In *Proceedings of the 40th International Conference on Software Engineering, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018*, Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman (Eds.). ACM, 433–444. https://doi.org/10.1145/3180155.3180164

[5] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding flaky tests: the developer's perspective. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2019)*. Association for Computing Machinery, New York, NY, USA, 830–840. https://doi.org/10.1145/3338906.3338945

[6] Aryaz Eghbali and Michael Pradel. 2022. DynaPyt: a dynamic analysis framework for Python. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*, Abhik Roychoudhury, Cristian Cadar, and Miryung Kim (Eds.). ACM, 760–771. https://doi.org/10.1145/3540250.3549126

[7] Vahid Garousi and Michael Felderer. 2016. Developing, Verifying, and Maintaining High-Quality Automated Test Scripts. *IEEE Softw.* 33, 3 (2016), 68–75. https://doi.org/10.1109/MS.2016.30

[8] Martin Gruber and Gordon Fraser. 2022. A Survey on How Test Flakiness Affects Developers and What Support They Need To Address It. In *15th IEEE Conference on Software Testing, Verification and Validation, ICST 2022, Valencia, Spain, April 4-14, 2022*. IEEE, 82–92. https://doi.org/10.1109/ICST53961.2022.00020

[9] Martin Gruber and Gordon Fraser. 2023. Debugging Flaky Tests using Spectrum-based Fault Localization. In *IEEE/ACM International Conference on Automation of Software Test, AST 2023, Melbourne, Australia, May 15-16, 2023*. IEEE, 128–139. https://doi.org/10.1109/AST58925.2023.00017

[10] Martin Gruber, Michael Heine, Norbert Oster, Michael Philippsen, and Gordon Fraser. 2023. Practical Flaky Test Prediction using Common Code Evolution and Test History Data. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2023, Dublin, Ireland, April 16-20, 2023*. IEEE, 210–221. https://doi.org/10.1109/ICST57152.2023.00028

[11] Martin Gruber, Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2021. An Empirical Study of Flaky Tests in Python. In *14th IEEE Conference on Software Testing, Verification and Validation, ICST 2021, Porto de Galinhas, Brazil, April 12-16, 2021*. IEEE, 148–158. https://doi.org/10.1109/ICST49551.2021.00026

[12] Sarra Habchi, Guillaume Haben, Mike Papadakis, Maxime Cordy, and Yves Le Traon. 2022. A Qualitative Study on the Sources, Impacts, and Mitigation Strategies of Flaky Tests. In *15th IEEE Conference on Software Testing, Verification and Validation, ICST 2022, Valencia, Spain, April 4-14, 2022*. IEEE, 244–255. https://doi.org/10.1109/ICST53961.2022.00034

[13] Sarra Habchi, Guillaume Haben, Jeongju Sohn, Adriano Franci, Mike Papadakis, Maxime Cordy, and Yves Le Traon. 2022. What Made This Test Flake? Pinpointing Classes Responsible for Test Flakiness. In *IEEE International Conference on Software Maintenance and Evolution, ICSME 2022, Limassol, Cyprus, October 3-7, 2022*. IEEE, 352–363. https://doi.org/10.1109/ICSME55016.2022.00039

[14] Guillaume Haben, Sarra Habchi, John Micco, Mark Harman, Mike Papadakis, Maxime Cordy, and Yves Le Traon. 2024. The Importance of Accounting for Execution Failures when Predicting Test Flakiness. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*, Vladimir Filkov, Baishakhi Ray, and Minghui Zhou (Eds.). ACM, 1979–1989. https://doi.org/10.1145/3691620.3695261

[15] Guillaume Haben, Sarra Habchi, Mike Papadakis, Maxime Cordy, and Yves Le Traon. 2021. A Replication Study on the Usability of Code Vocabulary in Predicting Flaky Tests. In *18th IEEE/ACM International Conference on Mining Software Repositories, MSR 2021, Madrid, Spain, May 17-19, 2021*. IEEE, 219–229. https://doi.org/10.1109/MSR52588.2021.00034

[16] IEEE Computer Society. 2025. *Guide to the Software Engineering Body of Knowledge v4.0*. IEEE Computer Society, United States.

[17] Tariq M. King, Dionny Santiago, Justin Phillips, and Peter J. Clarke. 2018. Towards a Bayesian Network Model for Predicting Flaky Automated Tests. In *2018 IEEE International Conference on Software Quality, Reliability and Security Companion, QRS Companion 2018, Lisbon, Portugal, July 16-20, 2018*. IEEE, 100–107. https://doi.org/10.1109/QRS-C.2018.00031

[18] Donald E. Knuth. 1984. Literate Programming. *Comput. J.* 27, 2 (1984), 97–111. https://doi.org/10.1093/COMJNL/27.2.97

[19] Wing Lam, Patrice Godefroid, Suman Nath, Anirudh Santhiar, and Suresh Thummalapenta. 2019. Root Causing Flaky Tests in a Large-Scale Industrial Setting. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, New York, NY,, 101–111. https://doi.org/10.1145/3293882.3330570

[20] Wing Lam, Kivanç Muslu, Hitesh Sajnani, and Suresh Thummalapenta. 2020. A study on the lifecycle of flaky tests. In *ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*, Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 1471–1482. https://doi.org/10.1145/3377811.3381749

[21] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests. In *12th IEEE Conference on Software Testing, Validation and Verification, ICST 2019, Xi'an, China, April 22-27, 2019*. IEEE, 312–322. https://doi.org/10.1109/ICST.2019.00038

[22] Wing Lam, Stefan Winter, Angello Astorga, Victoria Stodden, and Darko Marinov. 2020. Understanding Reproducibility and Characteristics of Flaky Tests Through Test Reruns in Java Projects. In *31st IEEE International Symposium on Software Reliability Engineering, ISSRE 2020, Coimbra, Portugal, October 12-15, 2020*, Marco Vieira, Henrique Madeira, Nuno Antunes, and Zheng Zheng (Eds.). IEEE, 403–413. https://doi.org/10.1109/ISSRE5003.2020.00045

[23] Johannes Lampel, Sascha Just, Sven Apel, and Andreas Zeller. 2021. When life gives you oranges: detecting and diagnosing intermittent job failures at Mozilla. In *ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, August 23-28, 2021*, Diomidis Spinellis, Georgios Gousios, Marsha Chechik, and Massimiliano Di Penta (Eds.). ACM, 1381–1392. https://doi.org/10.1145/3468264.3473931

[24] Fabian Leinen, Daniel Elsner, Alexander Pretschner, Andreas Stahlbauer, Michael Sailer, and Elmar Jürgens. 2024. Cost of Flaky Tests in Continuous Integration: An Industrial Case Study. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2024, Toronto, ON, Canada, May 27-31, 2024*. IEEE, 329–340. https://doi.org/10.1109/ICST60714.2024.00037

[25] Jeff Listfield. 2017. Google testing blog: Where do our flaky tests come from? https://testing.googleblog.com/2017/04/where-do-our-flaky-tests-come-from.html.

[26] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, (FSE-22), Hong Kong, China, November 16 - 22, 2014*, Shing-Chi Cheung, Alessandro Orso, and Margaret-Anne D. Storey (Eds.). ACM, 643–653. https://doi.org/10.1145/2635868.2635920

[27] Mateusz Machalica, Wojtek Chmiel, Stanislaw Swierc, and Ruslan Sakevych. 2020. Engineering at Meta: How do you test your tests? https://engineering.fb.com/2020/12/10/developer-tools/probabilistic-flakiness/.

[28] Aditya P. Mathur. 2008. *Foundations of Software Testing* (1st ed.). Addison-Wesley Professional.

[29] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A Survey of Flaky Tests. *ACM Trans. Softw. Eng. Methodol.* 31, 1, Article 17 (oct 2021), 74 pages. https://doi.org/10.1145/3476105

[30] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2022. Evaluating Features for Machine Learning Detection of Order- and Non-Order-Dependent Flaky Tests. In *15th IEEE Conference on Software Testing, Verification and Validation, ICST 2022, Valencia, Spain, April 4-14, 2022*. IEEE, 93–104. https://doi.org/10.1109/ICST53961.2022.00021

[31] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2022. Evaluating Features for Machine Learning Detection of Order- and Non-Order-Dependent Flaky Tests. In *15th IEEE Conference on Software Testing, Verification and Validation, ICST 2022, Valencia, Spain, April 4-14, 2022*. IEEE, 93–104. https://doi.org/10.1109/ICST53961.2022.00021

[32] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2022. Surveying the Developer Experience of Flaky Tests. In *44th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2022, Pittsburgh, PA, USA, May 22-24, 2022*. IEEE, 253–262. https://doi.org/10.1109/ICSE-SEIP55303.2022.9793965

[33] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2025. Systemic Flakiness: An Empirical Analysis of Co-Occurring Flaky Test Failures. *CoRR* abs/2504.16777 (2025). https://doi.org/10.48550/ARXIV.2504.16777 arXiv:2504.16777

[34] Juan Altmayer Pizzorno and Emery D. Berger. 2023. SlipCover: Near Zero-Overhead Code Coverage for Python. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*, René Just and Gordon Fraser (Eds.). ACM, 1195–1206. https://doi.org/10.1145/3597926.3598128

[35] Valeria Pontillo, Fabio Palomba, and Filomena Ferrucci. 2024. Test Code Flakiness in Mobile Apps: The Developer's Perspective. *Inf. Softw. Technol.* 168 (2024), 107394. https://doi.org/10.1016/J.INFSOF.2023.107394

[36] Alan Romano, Zihe Song, Sampath Grandhi, Wei Yang, and Weihang Wang. 2021. An Empirical Analysis of UI-based Flaky Tests. In *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid, Spain, 22-30 May 2021*. IEEE, 1585–1597. https://doi.org/10.1109/ICSE43902.2021.00141

[37] Amjed Tahir, Shawn Rasheed, Jens Dietrich, Negar Hashemi, and Lu Zhang. 2023. Test flakiness' causes, detection, impact and responses: A multivocal review. *J. Syst. Softw.* 206 (2023), 111837. https://doi.org/10.1016/J.JSS.2023.111837

[38] Swapna Thorve, Chandani Sreshtha, and Na Meng. 2018. An Empirical Study of Flaky Tests in Android Apps. In *2018 IEEE International Conference on Software*

Maintenance and Evolution, ICSME 2018, Madrid, Spain, September 23-29, 2018. IEEE Computer Society, 534–538. https://doi.org/10.1109/ICSME.2018.00062

[39] Jiaguo Wang, Yan Lei, Maojin Li, Guanyu Ren, Huan Xie, Shifeng Jin, Junchao Li, and Jian Hu. 2024. Flakyrank: Predicting Flaky Tests Using Augmented Learning to Rank. In *IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2024, Rovaniemi, Finland, March 12-15, 2024*. IEEE, 872–883. https://doi.org/10.1109/SANER60148.2024.00095

[40] Wei Zheng, Guoliang Liu, Manqing Zhang, Xiang Chen, and Wenqiao Zhao. 2021. Research Progress of Flaky Tests. In *28th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2021, Honolulu, HI, USA, March 9-12, 2021*. IEEE, 639–646. https://doi.org/10.1109/SANER50967.2021.00081

[41] Celal Ziftci and Diego Cavalcanti. 2020. De-Flake Your Tests : Automatically Locating Root Causes of Flaky Tests in Code At Google. In *IEEE International Conference on Software Maintenance and Evolution, ICSME 2020, Adelaide, Australia, September 28 - October 2, 2020*. IEEE, 736–745. https://doi.org/10.1109/ICSME46990.2020.00083